

MecâniQA — Fase 1: Explorando Estruturas de Dados Estáticas

Autor: Victor Paulo Teixeira da Silva, Luis Filipe Silva Lima, Guilherme Pereira Barreto, Brendha Maria Costa Seixas, Matheus da Costa Pereira, Agson Asafe

Professor: Lucas Almeida Silva

Agenda

O objetivo desta apresentação é demonstrar o MVP da Fase 1 do sistema MecâniQA, cobrindo a modelagem dos TADs, a estrutura de dados estática escolhida, a implementação do CRUD e os resultados da validação de qualidade (QA) aplicada ao código.

1. Contextualização do problema:

cenário da oficina mecânica MecâniQA e objetivo do MVP da Fase 1.

4. Resultados dos Testes de QA:

casos que passaram, casos negados e por quê.

2. TADs e Estruturas de Dados Estáticas:

modelagem de Peça e Serviço como structs e uso de arrays de tamanho fixo.

5. Considerações finais:

achados principais e próximos passos.

3. Listas Estáticas — CRUD e Busca Linear:

inserção, remoção, gestão de índices e busca sequencial.

- **O problema:** a oficina mecânica MecâniQA ainda controla peças em estoque e serviços prestados de forma manual, o que gera lentidão e risco de erro no cadastro e na consulta de dados.
- **O objetivo do MVP (Fase 1):** construir, em Java puro, um protótipo funcional de cadastro, consulta, atualização e remoção de Peças e Serviços usando estruturas de dados estáticas (arrays de tamanho fixo), sem banco de dados, para validar a lógica de negócio antes de evoluir para uma solução persistente.

TADs e Estruturas de Dados Estáticas

Itens para discussão:

- **Conceito de TAD:** Peça e Serviço foram modeladas como Tipos Abstratos de Dados na forma de structs — classes com apenas atributos públicos (código numérico, nome, fabricante, quantidade e preços para Peça; código, descrição, tempo estimado e valor de mão de obra para Serviço) e nenhuma regra de negócio interna.
- **Vantagens da alocação estática e acesso $O(1)$:** os arrays `pecas[100]` e `servicos[50]` ocupam um bloco contíguo de memória, o que garante acesso direto e instantâneo ($O(1)$) a qualquer posição pelo índice, sem a necessidade de percorrer a estrutura.
- **Limites físicos:** por ter tamanho fixo, a capacidade máxima é de 100 peças e 50 serviços; o cadastro de uma 101ª peça, por exemplo, precisa ser identificado e reportado ao usuário — ponto que os testes de QA mostraram estar ausente na versão atual do código.



Listas Estáticas: CRUD e Busca Linear

Itens para discussão:

- **Gestão de índices:** a inserção percorre o array até a primeira posição nula, sem uma variável auxiliar de controle de quantidade — o teste TC-07 apontou esse ponto como uma melhoria necessária para evitar buscas desnecessárias a cada cadastro.
- **Remoção e reorganização (operações virtuais):** a exclusão de Peça e Serviço desloca os elementos seguintes uma posição à esquerda, eliminando "buracos" no array e mantendo a lista compacta e sequencial.
- **Busca Linear e custo $O(n)$:** buscaLinearPeca e buscaLinearServico percorrem o array sequencialmente comparando o código informado; os testes revelaram que a busca ainda não retorna um índice reaproveitável, que updatePeca/updateServico repetem o mesmo laço em vez de reutilizar a busca, e identificaram um bug crítico em buscaLinearServico (TC-11).



Pontos de interesse:

- A validação de QA aplicada sobre Gerenciador.java executou 14 casos de teste: **5 passaram e 9 foram negados**, revelando pontos de melhoria concentrados em validação de entrada, controle de índices, limite de capacidade e reaproveitamento da busca linear (princípio DRY).
- O achado mais crítico (TC-11) mostra uma comparação lógica incorreta em buscaLinearServico, que compara o código do serviço com o tamanho do array e gera um erro contraditório mesmo quando o item é encontrado.
- Próximos passos da Fase 1: adicionar variável de controle de quantidade, validar a capacidade máxima antes da inserção, padronizar o retorno das buscas lineares como int (índice ou -1) e reaproveitar esse retorno nos métodos de atualização, eliminando a duplicação de código.

Referências do projeto:

- Material de apoio das aulas de Estrutura de Dados — UNEX, curso de Sistemas de Informação.
- Documentação oficial da linguagem Java (Oracle Java SE Documentation).
- Relatório de Testes de QA — mecaniQA Vitória (Gerenciador.java), Encontros 2 e 3.
- Baremas de avaliação — Estrutura de Dados, OAT 1, Vitória.